

✓ Prática: AlexNet com recursos limitados

Se antes implementamos a [AlexNet](#) sem considerar a questão de quantidade de parâmetros, nesta prática focaremos nesse quesito.

```
import time, os, sys, numpy as np
import torch
import torchvision
import torch.nn as nn
import torch.nn.functional as F

from torch import optim
from torchsummary import summary

import time, os, sys, numpy as np

# Test if GPU is available, if not, use cpu instead
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
n = torch.cuda.device_count()
devices_ids= list(range(n))
print(device)
```

cuda

```
def load_data_cifar10(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Cifar10-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.CIFAR10(root=root, train=True,download=True,transform=transformer)
    mnist_test = torchvision.datasets.CIFAR10(root=root, train=False,download=True,transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size, shuffle=False,
                                            num_workers=num_workers)

    return train_iter, test_iter

def load_data_fashion_mnist(batch_size, resize=None, root=os.path.join(
    '~', '.pytorch', 'datasets', 'fashion-mnist')):
    """Download the Fashion-MNIST dataset and then load into memory."""
    root = os.path.expanduser(root)
    transformer = []
    if resize:
        transformer += [torchvision.transforms.Resize(resize)]
    transformer += [torchvision.transforms.ToTensor()]
    transformer = torchvision.transforms.Compose(transformer)

    mnist_train = torchvision.datasets.FashionMNIST(root=root, train=True,download=True,transform=transformer)
    mnist_test = torchvision.datasets.FashionMNIST(root=root, train=False,download=True,transform=transformer)
    num_workers = 0 if sys.platform.startswith('win32') else 4

    train_iter = torch.utils.data.DataLoader(mnist_train,
                                              batch_size, shuffle=True,
                                              num_workers=num_workers)
    test_iter = torch.utils.data.DataLoader(mnist_test,
                                            batch_size, shuffle=False,
                                            num_workers=num_workers)

    return train_iter, test_iter

# funções básicas
def _get_batch(batch):
    """Return features and labels on ctx."""
    features, labels = batch
    if labels.type() != features.type():
        labels = labels.type(features.type())
    return (torch.nn.DataParallel(features, device_ids=devices_ids),
            torch.nn.DataParallel(labels, device_ids=devices_ids), features.shape[0])

# Função usada para calcular acurácia
```

```
def evaluate_accuracy(data_iter, net, loss):
    """Evaluate accuracy of a model on the given data set."""

    acc_sum, n, l = torch.Tensor([0]), 0, 0
    net.eval()
    with torch.no_grad():
        for X, y in data_iter:
            #y = y.astype('float32')
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l += loss(y_hat, y).sum()
            acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]

    return acc_sum.item() / n, l.item() / len(data_iter)

# Função usada no treinamento e validação da rede
def train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
                  num_epochs):
    print('training on', device)
    for epoch in range(num_epochs):
        net.train()
        train_l_sum, train_acc_sum, n, start = 0.0, 0.0, 0, time.time()
        for X, y in train_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            trainer.zero_grad()
            l = loss(y_hat, y).sum()
            l.backward()
            trainer.step()
            train_l_sum += l.item()
            train_acc_sum += (y_hat.argmax(axis=1) == y).sum().item()
            n += y.size()[0]
        test_acc, test_loss = evaluate_accuracy(test_iter, net, loss)
        print('epoch %d, train loss %.4f, train acc %.3f, test loss %.4f, '
              'test acc %.3f, time %.1f sec'
              % (epoch + 1, train_l_sum / len(train_iter), train_acc_sum / n, test_loss,
                 test_acc, time.time() - start))
```

✓ AlexNet

Como vimos, a rede [AlexNet](#) foi uma das arquiteturas mais famosas dessa nova onda de rede neurais.

AlexNet Network - Structural Details													
Input			Output			Layer	Stride	Pad	Kernel size		in	out	# of Param
227	227	3	55	55	96	conv1	4	0	11	11	3	96	34944
55	55	96	27	27	96	maxpool1	2	0	3	3	96	96	0
27	27	96	27	27	256	conv2	1	2	5	5	96	256	614656
27	27	256	13	13	256	maxpool2	2	0	3	3	256	256	0
13	13	256	13	13	384	conv3	1	1	3	3	256	384	885120
13	13	384	13	13	384	conv4	1	1	3	3	384	384	1327488
13	13	384	13	13	256	conv5	1	1	3	3	384	256	884992
13	13	256	6	6	256	maxpool5	2	0	3	3	256	256	0
						fc6			1	1	9216	4096	37752832
						fc7			1	1	4096	4096	16781312
						fc8			1	1	4096	1000	4097000
Total													62,378,344

Entretanto, ela possui muitos parâmetros. Especificamente, essa arquitetura, para classificar 10 classes, tem um total de **58.312.736** de parâmetros como mostrado na tabela abaixo.

Camada	Calc Parâmetros	Total Parâmetros
Convolução 1	11*11*3*96	34.848
Convolução 2	5*5*96*256	614.400
Convolução 3	3*3*256*384	884.736
Convolução 4	3*3*384*384	1.327.104
Convolução 5	3*3*384*256	884.736
FC 6	9216*4096	37.748.736
FC 7	4096*4096	16.777.216
FC 8	4096*10	40.960
Total		58.312.736

Seu objetivo nessa prática é propor uma nova rede neural, baseada na [AlexNet](#), que possuía MENOS parâmetros e alcance uma acurácia similar ou melhor que a rede original vista na aula passada.

Procure usar [batch normalization](#), camadas dilatadas e separáveis. Neste caso, desconsidere os parâmetros da camada [batch normalization](#).

✓ Rede original para efeito de comparação

```
class AlexNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):
        super(AlexNet, self).__init__(**kwargs)
        self.features = nn.Sequential(
            nn.Conv2d(in_channels=input_channels, out_channels=96, kernel_size=11, stride=4, padding=0),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0),

            nn.Conv2d(in_channels=96, out_channels=256, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0),

            nn.Conv2d(in_channels=256, out_channels=384, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),

            nn.Conv2d(in_channels=384, out_channels=384, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),

            nn.Conv2d(in_channels=384, out_channels=256, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),

            nn.Linear(9216, 4096),
            nn.ReLU(),
            nn.Dropout(0.5),

            nn.Linear(4096, 4096),
            nn.ReLU(),
            nn.Dropout(0.5),

            nn.Linear(4096, classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

```
num_epochs, lr, batch_size, wd_lambda = 20, 0.001, 100, 0.0001

net = AlexNet(3, 10)

# Sending model to device
net.to(device)
print(summary(net,(3,227,227))) # visualize number of parameters' net, output of each layer and total mega bytes necessary
                                # and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# trainer do gluon
trainer = optim.Adam(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 96, 55, 55]	34,944
ReLU-2	[-1, 96, 55, 55]	0
MaxPool2d-3	[-1, 96, 27, 27]	0
Conv2d-4	[-1, 256, 27, 27]	614,656
ReLU-5	[-1, 256, 27, 27]	0
MaxPool2d-6	[-1, 256, 13, 13]	0
Conv2d-7	[-1, 384, 13, 13]	885,120
ReLU-8	[-1, 384, 13, 13]	0

```

Conv2d-9      [-1, 384, 13, 13]      1,327,488
ReLU-10       [-1, 384, 13, 13]      0
Conv2d-11     [-1, 256, 13, 13]      884,992
ReLU-12       [-1, 256, 13, 13]      0
MaxPool2d-13  [-1, 256, 6, 6]      0
Flatten-14    [-1, 9216]      0
Linear-15     [-1, 4096]      37,752,832
ReLU-16       [-1, 4096]      0
Dropout-17    [-1, 4096]      0
Linear-18     [-1, 4096]      16,781,312
ReLU-19       [-1, 4096]      0
Dropout-20    [-1, 4096]      0
Linear-21     [-1, 10]      40,970
=====
Total params: 58,322,314
Trainable params: 58,322,314
Non-trainable params: 0
-----
Input size (MB): 0.59
Forward/backward pass size (MB): 11.11
Params size (MB): 222.48
Estimated Total Size (MB): 234.18
-----
None
Files already downloaded and verified
Files already downloaded and verified
training on cuda
epoch 1, train loss 1.9995, train acc 0.247, test loss 1.6608, test acc 0.385, time 92.1 sec
epoch 2, train loss 1.5566, train acc 0.427, test loss 1.4259, test acc 0.473, time 92.1 sec
epoch 3, train loss 1.3788, train acc 0.502, test loss 1.2590, test acc 0.547, time 92.2 sec
epoch 4, train loss 1.2490, train acc 0.552, test loss 1.1405, test acc 0.595, time 92.3 sec
epoch 5, train loss 1.1384, train acc 0.593, test loss 1.0679, test acc 0.622, time 91.8 sec
epoch 6, train loss 1.0558, train acc 0.628, test loss 1.0135, test acc 0.646, time 92.0 sec
epoch 7, train loss 0.9778, train acc 0.654, test loss 0.9917, test acc 0.655, time 91.8 sec
epoch 8, train loss 0.9199, train acc 0.674, test loss 0.9409, test acc 0.667, time 91.8 sec
epoch 9, train loss 0.8683, train acc 0.697, test loss 0.9345, test acc 0.678, time 91.5 sec
epoch 10, train loss 0.8216, train acc 0.714, test loss 0.8946, test acc 0.689, time 91.5 sec
epoch 11, train loss 0.7857, train acc 0.726, test loss 0.9312, test acc 0.680, time 92.1 sec
epoch 12, train loss 0.7416, train acc 0.742, test loss 0.8685, test acc 0.699, time 91.8 sec
epoch 13, train loss 0.7087, train acc 0.752, test loss 0.8866, test acc 0.697, time 92.3 sec
epoch 14, train loss 0.6769, train acc 0.761, test loss 0.8813, test acc 0.699, time 91.6 sec
epoch 15, train loss 0.6542, train acc 0.770, test loss 0.8705, test acc 0.703, time 92.1 sec
epoch 16, train loss 0.6257, train acc 0.780, test loss 0.8911, test acc 0.706, time 91.6 sec
epoch 17, train loss 0.6060, train acc 0.789, test loss 0.8816, test acc 0.710, time 91.9 sec
epoch 18, train loss 0.5723, train acc 0.799, test loss 0.8914, test acc 0.714, time 92.0 sec
epoch 19, train loss 0.5569, train acc 0.805, test loss 0.8953, test acc 0.706, time 92.8 sec
epoch 20, train loss 0.5382, train acc 0.811, test loss 0.9188, test acc 0.701, time 93.4 sec

```

Arquitetura 1

Essa primeira versão da [AlexNet](#) usa [batch normalization](#) para melhorar o resultado. Esse arquitetura é somente para efeitos de comparação já que, por enquanto, ainda temos a mesma quantidade de parâmetros da versão original.

```

class AlexNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):
        super(AlexNet, self).__init__(**kwargs)
        self.features = nn.Sequential(
            nn.Conv2d(in_channels=input_channels, out_channels=96, kernel_size=11, stride=4, padding=0),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=96),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0),

            nn.Conv2d(in_channels=96, out_channels=256, kernel_size=5, stride=1, padding=2),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=256),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0),

            nn.Conv2d(in_channels=256, out_channels=384, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=384),

            nn.Conv2d(in_channels=384, out_channels=384, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=384),

            nn.Conv2d(in_channels=384, out_channels=256, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=256),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),

            nn.Linear(9216, 4096),

```

```

        nn.ReLU(),
        nn.BatchNorm1d(num_features=4096),
        nn.Dropout(0.5),

        nn.Linear(4096, 4096),
        nn.ReLU(),
        nn.BatchNorm1d(num_features=4096),
        nn.Dropout(0.5),

        nn.Linear(4096, classes)
    )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

```

```

num_epochs, lr, batch_size, wd_lambda = 20, 0.001, 100, 0.0001

net = AlexNet(3, 10)

# Sending model to device
net.to(device)
print(summary(net,(3,227,227))) # visualize number of parameters' net, output of each layer and total mega bytes necessary
                                # and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()

# carregamento do dado: mnist
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# trainer do gluon
trainer = optim.Adam(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)

```

Layer (type)	Output Shape	Param #
-----	-----	-----
Conv2d-1	[-1, 96, 55, 55]	34,944
ReLU-2	[-1, 96, 55, 55]	0
BatchNorm2d-3	[-1, 96, 55, 55]	192
MaxPool2d-4	[-1, 96, 27, 27]	0
Conv2d-5	[-1, 256, 27, 27]	614,656
ReLU-6	[-1, 256, 27, 27]	0
BatchNorm2d-7	[-1, 256, 27, 27]	512
MaxPool2d-8	[-1, 256, 13, 13]	0
Conv2d-9	[-1, 384, 13, 13]	885,120
ReLU-10	[-1, 384, 13, 13]	0
BatchNorm2d-11	[-1, 384, 13, 13]	768
Conv2d-12	[-1, 384, 13, 13]	1,327,488
ReLU-13	[-1, 384, 13, 13]	0
BatchNorm2d-14	[-1, 384, 13, 13]	768
Conv2d-15	[-1, 256, 13, 13]	884,992
ReLU-16	[-1, 256, 13, 13]	0
BatchNorm2d-17	[-1, 256, 13, 13]	512
MaxPool2d-18	[-1, 256, 6, 6]	0
Flatten-19	[-1, 9216]	0
Linear-20	[-1, 4096]	37,752,832
ReLU-21	[-1, 4096]	0
BatchNorm1d-22	[-1, 4096]	8,192
Dropout-23	[-1, 4096]	0
Linear-24	[-1, 4096]	16,781,312
ReLU-25	[-1, 4096]	0
BatchNorm1d-26	[-1, 4096]	8,192
Dropout-27	[-1, 4096]	0
Linear-28	[-1, 10]	40,970
=====	=====	=====

Total params: 58,341,450
 Trainable params: 58,341,450
 Non-trainable params: 0

 Input size (MB): 0.59
 Forward/backward pass size (MB): 16.13
 Params size (MB): 222.55
 Estimated Total Size (MB): 239.28

None
 Files already downloaded and verified
 Files already downloaded and verified
 training on cuda
 epoch 1, train loss 1.8312, train acc 0.421, test loss 1.6312, test acc 0.464, time 97.5 sec
 epoch 2, train loss 1.5349, train acc 0.535, test loss 1.8795, test acc 0.433, time 97.6 sec

```
epoch 3, train loss 1.4115, train acc 0.590, test loss 3.7439, test acc 0.569, time 97.7 sec
epoch 4, train loss 1.3658, train acc 0.616, test loss 1.7238, test acc 0.585, time 97.4 sec
epoch 5, train loss 1.1965, train acc 0.657, test loss 1.8943, test acc 0.617, time 97.3 sec
epoch 6, train loss 1.1110, train acc 0.685, test loss 1.2236, test acc 0.666, time 97.7 sec
epoch 7, train loss 1.0765, train acc 0.701, test loss 1.1200, test acc 0.685, time 97.3 sec
epoch 8, train loss 1.0201, train acc 0.714, test loss 0.9428, test acc 0.741, time 97.1 sec
epoch 9, train loss 1.1125, train acc 0.691, test loss 0.9165, test acc 0.732, time 97.5 sec
epoch 10, train loss 0.9917, train acc 0.725, test loss 1.2072, test acc 0.706, time 97.2 sec
epoch 11, train loss 1.0551, train acc 0.710, test loss 1.7279, test acc 0.705, time 97.6 sec
epoch 12, train loss 0.8047, train acc 0.757, test loss 0.8602, test acc 0.763, time 97.2 sec
epoch 13, train loss 0.7271, train acc 0.765, test loss 1.6154, test acc 0.738, time 97.3 sec
```

Arquitetura 2

Essa segunda versão da [AlexNet](#) usa [batch normalization](#) e camadas de [convolução dilatada](#). Neste caso, como usamos filtros dilatados em duas camadas (com dilatação aumentante, ou seja, dilatação 2 seguida da dilatação 4), foram removidos duas camadas convolucionais, já que o *receptive field* se mantém similar dessa forma.

Nessa arquitetura, já temos menos parâmetros. Precisamente, temos

```
class AlexNet(nn.Module):
    def __init__(self, input_channels, classes=10, **kwargs):
        super(AlexNet, self).__init__(**kwargs)
        self.features = nn.Sequential(
            nn.Conv2d(in_channels=input_channels, out_channels=96, kernel_size=11, stride=4, padding=0),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=96),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0),

            nn.Conv2d(in_channels=96, out_channels=256, kernel_size=5, stride=1, padding=2, dilation=2),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=256),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0),

            # nn.Conv2d(in_channels=256, out_channels=384, kernel_size=3, stride=1, padding=1),
            # nn.ReLU(),
            # nn.BatchNorm2d(num_features=384),

            # nn.Conv2d(in_channels=384, out_channels=384, kernel_size=3, stride=1, padding=1),
            # nn.ReLU(),
            # nn.BatchNorm2d(num_features=384),

            nn.Conv2d(in_channels=256, out_channels=256, kernel_size=3, stride=1, padding=1, dilation=4),
            nn.ReLU(),
            nn.BatchNorm2d(num_features=256),
            nn.MaxPool2d(kernel_size=3, stride=2, padding=0)
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),

            nn.Linear(1024, 4096),
            nn.ReLU(),
            nn.BatchNorm1d(num_features=4096),
            nn.Dropout(0.5),

            nn.Linear(4096, 4096),
            nn.ReLU(),
            nn.BatchNorm1d(num_features=4096),
            nn.Dropout(0.5),

            nn.Linear(4096, classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

```
num_epochs, lr, batch_size, wd_lambda = 20, 0.001, 100, 0.0001

net = AlexNet(3, 10)

# Sending model to device
net.to(device)
print(summary(net,(3,227,227))) # visualize number of parameters' net, output of each layer and total mega bytes necessary
# and stored weights.

# função de custo (ou loss)
loss = nn.CrossEntropyLoss()
```

```
# carregamento do dado: mnist
train_iter, test_iter = load_data_cifar10(batch_size, resize=227)

# trainer do gluon
trainer = optim.Adam(net.parameters(), lr=lr, weight_decay=wd_lambda)

# treinamento e validação via Pytorch
train_validate(net, train_iter, test_iter, batch_size, trainer, loss,
               num_epochs)
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 96, 55, 55]	34,944
ReLU-2	[-1, 96, 55, 55]	0
BatchNorm2d-3	[-1, 96, 55, 55]	192
MaxPool2d-4	[-1, 96, 27, 27]	0
Conv2d-5	[-1, 256, 23, 23]	614,656
ReLU-6	[-1, 256, 23, 23]	0
BatchNorm2d-7	[-1, 256, 23, 23]	512
MaxPool2d-8	[-1, 256, 11, 11]	0
Conv2d-9	[-1, 256, 5, 5]	590,080
ReLU-10	[-1, 256, 5, 5]	0
BatchNorm2d-11	[-1, 256, 5, 5]	512
MaxPool2d-12	[-1, 256, 2, 2]	0
Flatten-13	[-1, 1024]	0
Linear-14	[-1, 4096]	4,198,400
ReLU-15	[-1, 4096]	0
BatchNorm1d-16	[-1, 4096]	8,192
Dropout-17	[-1, 4096]	0
Linear-18	[-1, 4096]	16,781,312
ReLU-19	[-1, 4096]	0
BatchNorm1d-20	[-1, 4096]	8,192
Dropout-21	[-1, 4096]	0
Linear-22	[-1, 10]	40,970

=====
Total params: 22,277,962

Trainable params: 22,277,962

Non-trainable params: 0

Input size (MB): 0.59

Forward/backward pass size (MB): 10.93

Params size (MB): 84.98

Estimated Total Size (MB): 96.50

None

Files already downloaded and verified

Files already downloaded and verified

training on cuda

```
epoch 1, train loss 1.6600, train acc 0.467, test loss 1.4030, test acc 0.546, time 98.8 sec
epoch 2, train loss 1.2137, train acc 0.615, test loss 1.1222, test acc 0.612, time 99.2 sec
epoch 3, train loss 0.9860, train acc 0.688, test loss 1.0605, test acc 0.678, time 99.2 sec
epoch 4, train loss 0.8627, train acc 0.726, test loss 0.9006, test acc 0.709, time 98.0 sec
epoch 5, train loss 0.6815, train acc 0.769, test loss 0.8443, test acc 0.726, time 97.3 sec
epoch 6, train loss 0.5663, train acc 0.807, test loss 3.2595, test acc 0.727, time 97.1 sec
epoch 7, train loss 0.4898, train acc 0.831, test loss 0.8398, test acc 0.746, time 96.6 sec
epoch 8, train loss 0.4206, train acc 0.855, test loss 0.8171, test acc 0.754, time 96.4 sec
epoch 9, train loss 0.3607, train acc 0.877, test loss 0.8848, test acc 0.732, time 95.9 sec
epoch 10, train loss 0.3189, train acc 0.891, test loss 0.9928, test acc 0.724, time 96.4 sec
epoch 11, train loss 0.2605, train acc 0.910, test loss 0.9701, test acc 0.733, time 96.3 sec
epoch 12, train loss 0.2199, train acc 0.925, test loss 0.9351, test acc 0.748, time 96.1 sec
epoch 13, train loss 0.2077, train acc 0.928, test loss 0.9760, test acc 0.754, time 96.1 sec
epoch 14, train loss 0.1819, train acc 0.938, test loss 1.0235, test acc 0.741, time 96.2 sec
epoch 15, train loss 0.1730, train acc 0.941, test loss 0.9771, test acc 0.746, time 96.2 sec
epoch 16, train loss 0.1514, train acc 0.948, test loss 1.0424, test acc 0.741, time 96.0 sec
epoch 17, train loss 0.1468, train acc 0.949, test loss 1.1505, test acc 0.711, time 96.0 sec
epoch 18, train loss 0.1383, train acc 0.951, test loss 1.0512, test acc 0.746, time 96.0 sec
epoch 19, train loss 0.1214, train acc 0.958, test loss 1.0535, test acc 0.757, time 96.1 sec
```

